

Payment Webhooks

Endpoint: `POST /webhooks/payment` (provider: **AbacatePay**).

Signature verification

- The route parses the **raw request body** (`express.raw`) — mounted before the global JSON parser — because signatures are computed over the exact bytes.
- Verification is **HMAC-SHA256** of the raw body keyed with `PAYMENT_WEBHOOK_SECRET`, compared in constant time. A `sha256=` prefix on the header is tolerated.
- A missing or invalid signature returns **401** and makes no state change.

The exact AbacatePay signature header/scheme is confirmed against their sandbox docs and adjusted only inside `src/payments/abacatepay.ts` (`verifyWebhook`). Everything downstream is provider-agnostic.

Idempotency

Every event is recorded in `WebhookEvent` with a unique (`provider`, `providerEventId`) index. A replayed or retried delivery hits that unique constraint and returns **200 duplicate_ignored** without reprocessing — so duplicate/retried webhooks **confirm once and charge once**. Decisions key off payment **status**, not arrival order, so out-of-order delivery is safe.

Exception matrix

All branches run inside one transaction that takes `SELECT ... FOR UPDATE` on the reservation **and** its slot — the same lock the expiry job uses — so the payment-at-expiry race always resolves to exactly one consistent outcome.

Event	Slot/reservation state	Action	Result
approved	slot <code>Reserved</code> , reservation <code>Active</code>	slot → <code>Paid</code> , reservation → <code>Paid</code> , payment → <code>Approved</code>	Booking confirmed
approved	slot already released / re-sold / expired	payment → <code>Refunded</code> , dispatch provider refund	Charge voided, never a phantom booking
rejected	any	payment → <code>Rejected</code>	Slot left to expire via its TTL
pending	any	no-op (logged)	Awaiting a terminal event

Event	Slot/reservation state	Action	Result
any (replay)	duplicate providerEventId	none	200 duplicate_ignored
invalid signature	—	none	401

Payment-at-expiry outcomes (both correct)

Outcome	Meaning
Slot Paid , reservation Paid , payment Approved	Webhook won the row lock first
Slot freed (Available / Expired), payment Refunded	Expiry won; charge auto-voided

The forbidden state — payment Approved but slot not Paid (charged with no booking) — is impossible because the confirm branch only fires while the slot is still Reserved under the lock. Proven across 50 concurrent rounds by `npm run verify:webhook`.

Refund dispatch

The refund (provider I/O) is dispatched **after** the transaction commits, so no row lock is held across a network call. If the refund call fails the payment is already marked Refunded and the failure is logged for manual/ops retry.

Production hardening (noted, not yet built): move refund dispatch to a durable pg-boss job with automatic retries.